

OS SIMULATOR - PROJECT DOCUMENTATION

- *Mrs. Krati Shrivastava*

Abstract

This Python-based Operating System Simulator provides an interactive desktop environment to visualize complex theoretical OS concepts. Utilizing a Tkinter interface, it dynamically demonstrates CPU Scheduling, the Banker's Algorithm for deadlock avoidance, and memory Page Replacement strategies, bridging the gap between abstract academic theory and observable system mechanics.

Chaitanya Kulkarni
25ET2041010[CSE]

Introduction

The OS Simulator is a comprehensive, Python-based desktop application designed to interactively demonstrate fundamental Operating System concepts. Built utilizing a graphical Tkinter interface, the software provides a highly visual and intuitive environment for exploring complex system behaviors. The simulator is primarily divided into three core educational modules: CPU Scheduling, Banker's Algorithm for deadlock avoidance, and Memory Page Replacement. By visualizing these historically abstract concepts, the tool bridges the gap between theoretical operating system textbook principles and practical, observable execution.

Technology Stack

The application is built entirely using open-source, industry-standard Python libraries to ensure cross-platform compatibility and ease of deployment.

Technology	Purpose / Usage in Project
Python	Core programming language utilized for all backend logic and algorithm implementation.
Tkinter	Standard GUI framework utilized to construct the desktop window environments and input forms.
Matplotlib	Data visualization library used to dynamically generate and render charts, such as the Gantt charts for CPU scheduling.

System Architecture & Project Structure

The project follows a modular design pattern, separating discrete operating system concepts into their own distinct Python modules. This allows for maintainability and clear separation of concerns. The directory tree below illustrates the application's source code organization:

```
os-simulator/
├── main.py
├── cpu_scheduling.py
├── bankers_algorithm.py
├── page_replacement.py
├── original_single_file.py
├── requirements.txt
├── .gitignore
└── assets/
    └── images/
        ├── cpu-scheduling.jpg
        ├── bankers-algorithm.jpg
        └── page-replacement.jpg
```

Note: The root file `main.py` manages the primary graphical interface and application routing, while the individual algorithm files contain specific operational logic. An `original_single_file.py` is maintained in the repository to serve as a legacy backup prior to the modular refactoring

Project Novelty & Utility

Operating system concepts often suffer from being overly theoretical and difficult to trace mentally, particularly for computer science students and early-career developers. The novelty of the OS Simulator lies in its highly interactive, visual-first approach to these historically opaque algorithms. Rather than relying on static textbook diagrams or dense command-line outputs, this application provides an immediate, graphical representation of state changes, memory allocations, and CPU utilization.

Key Utility and Benefits:

- **Interactive Visualization:** Gantt charts are generated dynamically for CPU scheduling, offering a real-time view of process turnaround and waiting times. This allows users to immediately compare the efficiency of algorithms like Round Robin versus FCFS under varying loads.
- **Safe-State Sandboxing:** The Banker's Algorithm module operates as a secure sandbox where users can test hypothetical resource allocation scenarios without risking actual system deadlocks. The explicit breakdown of the 'Need' matrix and the step-by-step derivation of the safe sequence demystify resource management.
- **Memory Traceability:** Memory replacement often involves tracking complex frame shifts. The application's step-by-step visualization of page hits, misses, and frame states transforms a tedious manual calculation into a clear, traceable animation of hardwarelevel memory management.

Ultimately, the OS Simulator serves as an invaluable educational utility. It empowers instructors to demonstrate real-time process management and enables students to intuitively grasp the performance trade-offs inherent in different architectural designs.

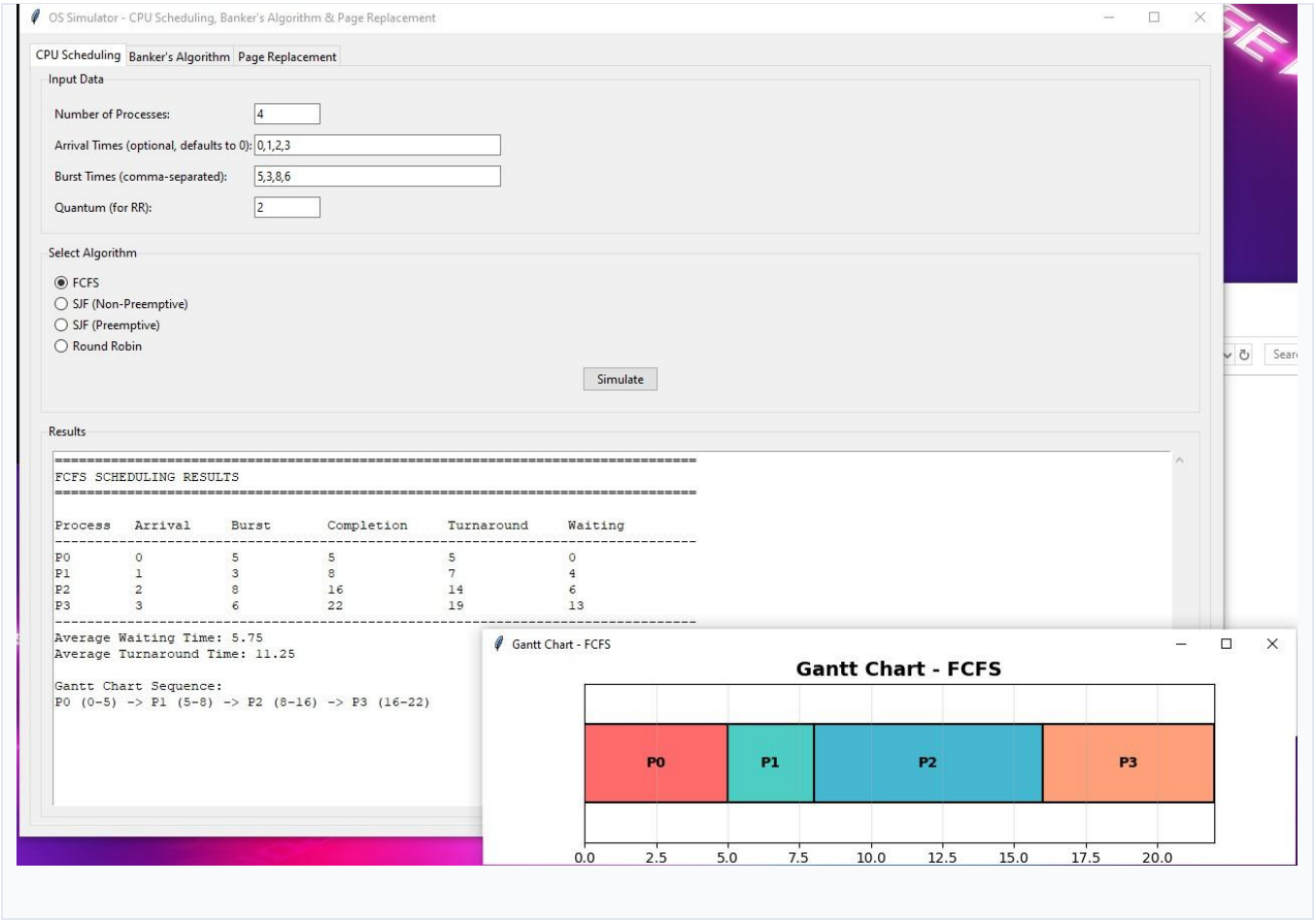
Functional Modules

CPU Scheduling

The CPU Scheduling module allows the user to input custom process parameters to simulate various process management algorithms utilized by modern operating systems. The tool calculates comprehensive performance metrics including Completion Time, Turnaround Time, Waiting Time, alongside Averages for Waiting and Turnaround times.

Supported algorithms include:

- **FCFS:** First Come First Serve processing.
- **SJF (Non-Preemptive):** Shortest Job First without task interruption.
- **SJF (Preemptive):** Shortest Remaining Time First.
- **Round Robin:** Time-sliced processing utilizing a defined time quantum.

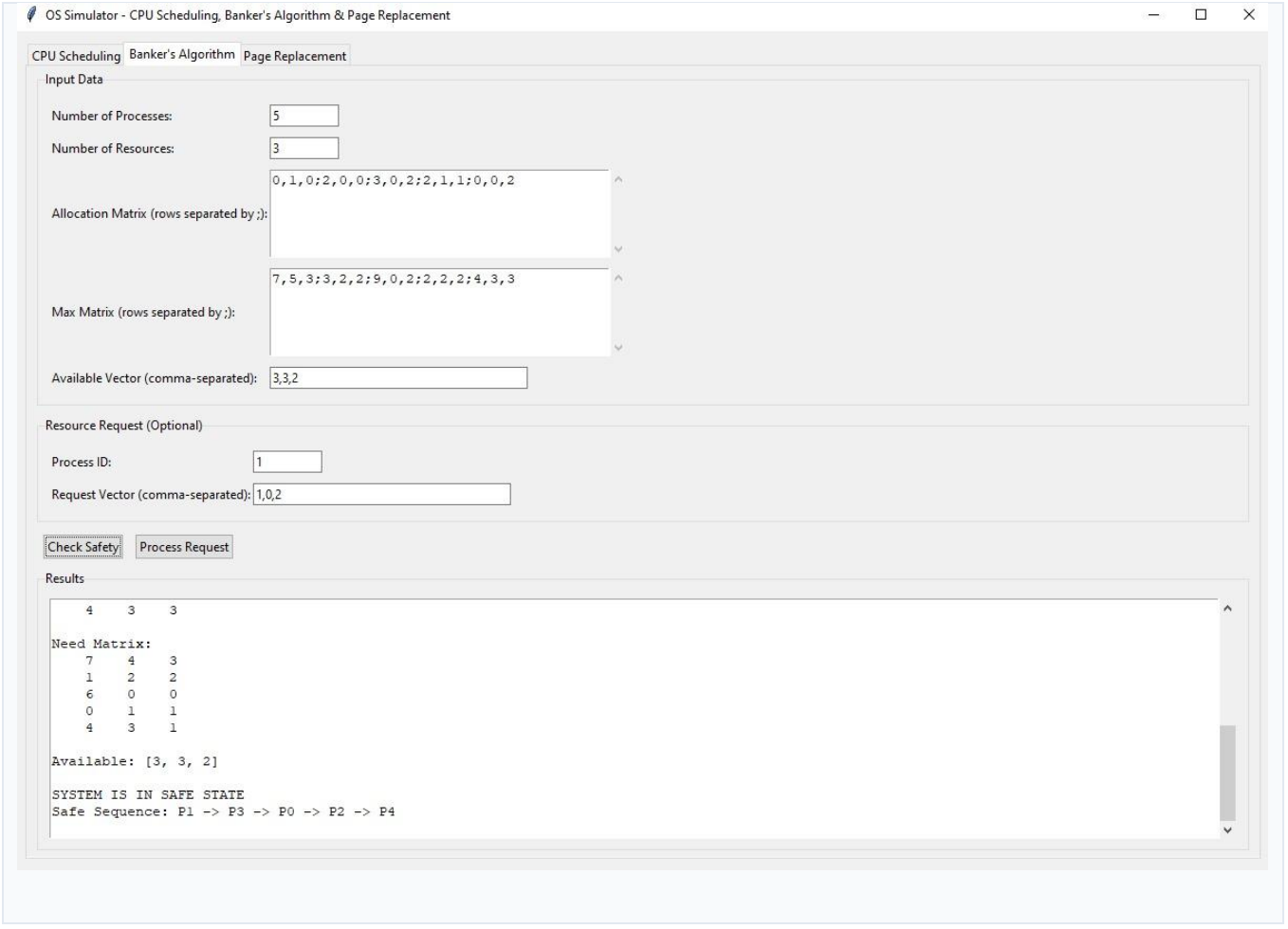


Banker's Algorithm

This module implements Dijkstra's Banker's Algorithm to evaluate whether a given system state is safe from deadlock. It bases its computation on user-provided Allocation matrices, Max matrices, and Available resource vectors. Furthermore, the module supports testing hypothetical resource requests from specific processes to verify if granting the request would result in a safe or unsafe system state.

The interface actively outputs:

- Calculated Need matrix and dynamic Available resources.
- A definitive assessment of Safe vs. Unsafe state.
- The validated Safe Sequence, if the system is determined to be secure.

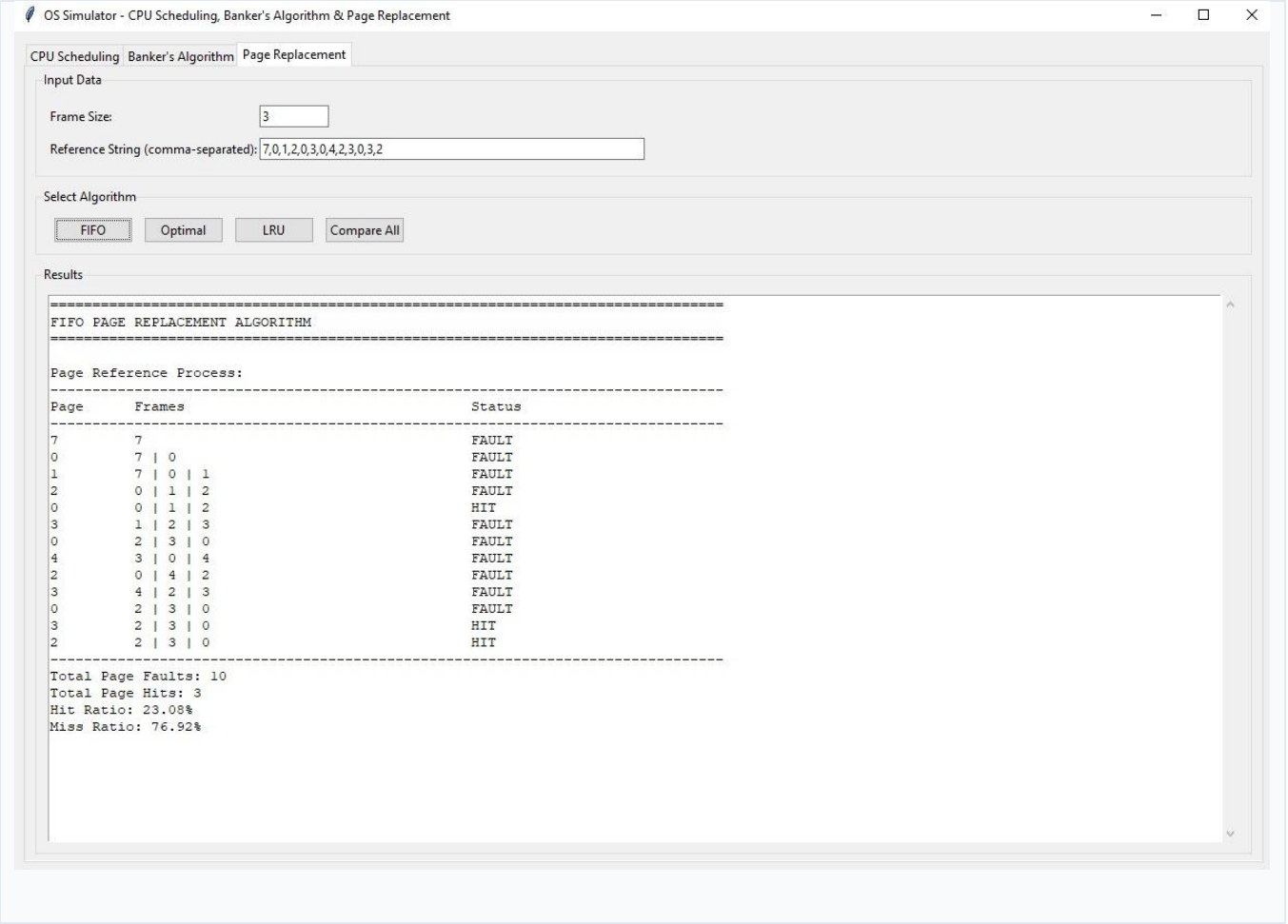


Memory Page Replacement

The Page Replacement simulator models how an operating system manages memory frames. By accepting a custom reference string and a defined frame size, the system steps through sequential memory requests and visualizes the frame changes step-by-step. The module automatically calculates Page Faults, Page Hits, Hit Ratio, and Miss Ratio.

Supported replacement algorithms include:

- **FIFO:** First In First Out.
- **Optimal:** Replaces the page that will remain unused for the longest future duration.
- **LRU:** Least Recently Used page replacement.
- **Compare All:** A unified view analyzing all three algorithms simultaneously for direct efficiency comparison.



Deployment & Execution Instructions

The application is designed to be easily initialized from a standard command-line interface. To run the simulator locally, follow the sequential steps below:

Step 1: Clone the remote repository to your local machine and navigate into the root directory.

```
git clone https://github.com/YOUR_USERNAME/YOUR_REPO.git cd YOUR_REPO
```

Step 2: Install the required Python dependencies to ensure Matplotlib and graphical components execute correctly.

```
pip install -r requirements.txt
```

Step 3: Launch the application interface via the main Python script.

```
python main.py
```

References

The concepts, algorithms, and methodologies implemented within the OS Simulator are supported by foundational literature in computer science and operating systems design. The following resources provide the theoretical basis for the application's core modules:

[1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
(Provides the primary theoretical foundation for CPU Scheduling and Memory Management).

[2] Tanenbaum, A. S., & Bos, H. (2014). *Modern Operating Systems* (4th ed.). Pearson.
(Reference for the detailed mechanics of Page Replacement Algorithms).

[3] Dijkstra, E. W. (1965). *Cooperating Sequential Processes*. Technological University Eindhoven.
(Original formulation of the Banker's Algorithm for deadlock avoidance).

[4] Python Software Foundation. (2023). *Tkinter — Python interface to Tcl/Tk*. Python 3.12 Documentation.
(Official documentation for the graphical interface framework).

[5] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95.
(Provides the framework for generating process execution Gantt charts).